

CSE321 Lab Term Project

Summer 2026

SimpleFS: Implementation of a Simple File System in C

1. Introduction:

A file system organizes and stores files on a storage device. Internally, it divides storage into blocks and maintains data structures to keep track of files, metadata, directories, and free space.

In this project, you will implement a small educational file system called SimpleFS using the C programming language. SimpleFS is based on the file-system implementation concepts covered in the course and includes a superblock, inode bitmap, data bitmap, inode table, root directory, and data blocks.

You will create and modify a binary file-system image that represents a small disk. You are not implementing a Linux-compatible file system, and you do not need to mount the image. The goal is to understand how the major structures of a file system work together.

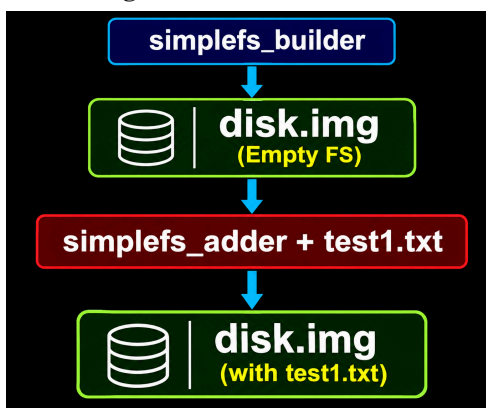
2. Objectives:

- Explain how a disk can be divided into fixed-size blocks.
- Explain the purpose of a superblock and an inode.
- Explain how inode and data bitmaps track allocated and free objects.
- Explain how a directory maps a file name to an inode number.
- Explain how an inode points to the data blocks of a file.
- Implement first-fit inode and data-block allocation using bitmaps.
- Store file contents inside a binary file-system image.
- Use **xxd** or **hexdump** to examine a binary image and verify correctness.

3. Implementation Goal:

You will implement two C programs:

- simplefs_builder.c** - creates and initializes an empty SimpleFS image.
- simplefs_adder.c** - adds a regular file from the current working directory to an existing SimpleFS image.



4. Project Files:

Each group will receive the following starter files:

```
simplefs.h
simplefs_builder.c
simplefs_adder.c

test1.txt
test2.txt
test3.txt
```

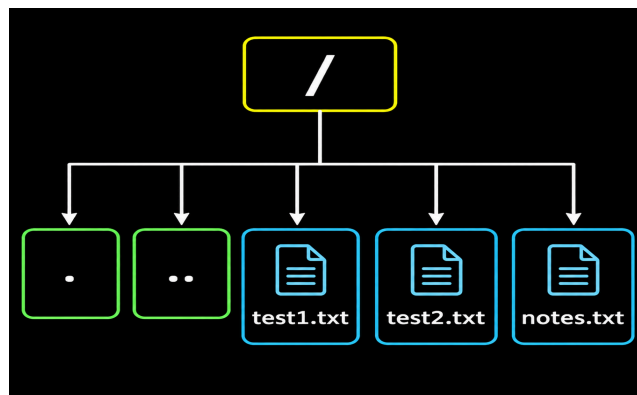
The source file contains sections marked `/* TODO: STUDENT CODE */`. You must complete those sections. The file system structure definitions and constants are provided in **simplefs.h** and must not be changed.

5. Simplifications Used in SimpleFS:

SimpleFS is intentionally limited so that the project focuses on the core file-system concepts. You do not need to implement the following:

- Subdirectories
- Indirect block pointers
- File deletion
- A separate file-read command
- File renaming
- Hard links or symbolic links
- Permissions
- Journaling
- Checksums
- File-system mounting
- Caching
- Multi-level directory traversal

Only the root directory (`/`) exists. All user files are stored directly inside it.



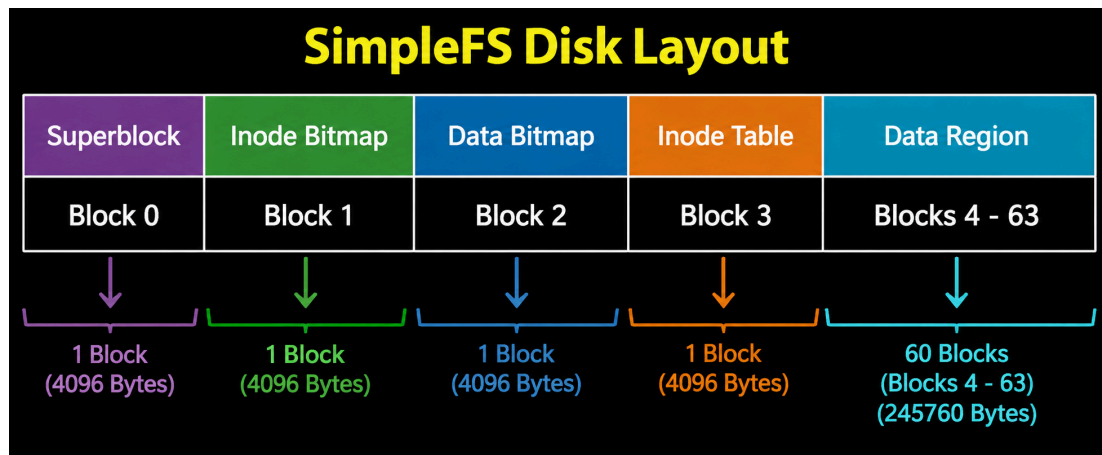
Source files must be placed in the current working directory. Path handling, such as **files/test1.txt** is not required.

6. Fixed SimpleFS Parameters:

Parameter	Value
Block Size	4096 bytes
Total Blocks	64
Total Image Size	262144 bytes (256 KiB)
Number of Inodes	32
Size of Each Inode	128 bytes
Direct Pointers per Inode	3
Maximum File Size	12288 bytes
Maximum File Name	58 characters
Root Inode Number	1
First Data Block	Block 4

Total image size = 64 x 4096 = 262144 bytes = 256 KiB

7. File-System Disk Layout:



Region	Block(s)	Blocks	Bytes Reserved	Starting Byte
Superblock	0	1	4,096	0
Inode Bitmap	1	1	4,096	4,096
Data Bitmap	2	1	4,096	8,192
Inode Table	3	1	4,096	12,288
Data Region	4-63	60	245,760	16,384
Total	0-63	64	262,144	-

- **Understanding Block Positions:**

Each block contains 4096 bytes. The starting byte of a block is calculated as:

Starting byte = Block number x 4096

Block	Calculation	Starting Byte
0	0 x 4096	0
1	1 x 4096	4096
2	2 x 4096	8192
3	3 x 4096	12288
4	4 x 4096	16384
5	5 x 4096	20480

fseek(fp, 4 * BLOCK_SIZE, SEEK_SET);

The example above moves the file pointer to the beginning of Block 4

8. Superblock:

The superblock describes the organization of the file system and is stored in Block 0.

```
typedef struct {
    uint32_t magic;
    uint32_t block_size;
    uint32_t total_blocks;
    uint32_t inode_count;

    uint32_t inode_bitmap_block;
    uint32_t data_bitmap_block;
    uint32_t inode_table_block;
    uint32_t data_region_block;

    uint32_t root_inode;
} superblock_t;
```

Field	Bytes	Required Value
magic	4	0x53465331
block_size	4	4096
total_blocks	4	64
inode_count	4	32
inode_bitmap_block	4	1
data_bitmap_block	4	2

Field	Bytes	Required Value
inode_table_block	4	3
data_region_block	4	4
root_inode	4	1
Actual structure size	36	
Remaining bytes in Block 0	4060	All zero
Reserved superblock region	4096	

Important: the superblock structure itself is 36 bytes, but the entire 4096-byte Block 0 is reserved for the superblock region. The remaining bytes must remain zero.

9. Inodes and the Inode Table:

An inode stores metadata about a file or directory. The inode does not store the file name. Each SimpleFS inode occupies exactly 128 bytes.

```
typedef struct {
    uint16_t type;
    uint16_t links;
    uint32_t size;
    uint32_t direct[3];
    uint8_t reserved[108];
} inode_t;
```

Field	Bytes	Purpose
type	2	File or directory
links	2	Link count
size	4	Actual size in bytes
direct[0]	4	First data-block number
direct[1]	4	Second data-block number
direct[2]	4	Third data-block number
reserved	108	Unused; keep zero
Total	128	

Use type = 1 for a regular file and type = 2 for a directory.

```
printf("%zu\n", sizeof(inode_t)); // must print 128
```

I. Inode Table Size and Location:

```
32 inodes x 128 bytes = 4096 bytes = exactly 1 block
```

The complete inode table, therefore, occupies Block 3.

Inode No.	Table Index	Offset in Block 3	Initial Purpose
1	0	0	Root directory
2	1	128	First available file inode
3	2	256	Available
4	3	384	Available
...	...	...	...
32	31	3968	Last inode

Absolute inode byte position = 12288 + (inode_number - 1) x 128

Example for inode 2:

12288 + (2 - 1) x 128 = 12416

II. Direct Block Pointers:

Each inode contains three direct pointers. The values stored in **direct[]** are absolute disk block numbers.

```
direct[0] = 5
direct[1] = 6
direct[2] = 0
```

This means the first part of the file is in Block 5, the second part is in Block 6, and the third direct pointer is unused. A direct pointer value of 0 means unused.

III. Maximum File Size:

Maximum file size = 3 direct blocks x 4096 bytes = 12288 bytes

Files larger than 12288 bytes must be rejected with an understandable error message.

10. Inode Bitmap:

The inode bitmap is stored in Block 1. Each bit indicates whether the corresponding inode is free or allocated: 0 means free and 1 means allocated.

```
Bit 0 -> Inode 1
Bit 1 -> Inode 2
Bit 2 -> Inode 3
...
Bit 31 -> Inode 32
```

Thirty-two inodes require only 32 bits = 4 bytes, but one complete 4096-byte block is reserved for the inode bitmap. The unused bytes remain zero. Immediately after

formatting, inode 1 belongs to the root directory, so Bit 0 must be 1, and all other inode bits must be 0.

11. Data Bitmap:

The data bitmap is stored in Block 2 and tracks the 60 blocks in the data region.

```
Bit 0 -> Absolute Block 4
Bit 1 -> Absolute Block 5
Bit 2 -> Absolute Block 6
Bit 3 -> Absolute Block 7
...
```

Sixty data blocks require 60 bits, so 8 bytes are sufficient to hold the meaningful bits. However, the whole 4096-byte Block 2 is reserved for the bitmap. Immediately after formatting, Bit 0 must be 1 because Block 4 is used by the root directory.

- **Bitmap Operations:**

```
// Mark a bitmap bit as allocated
bitmap[index / 8] |= (1 << (index % 8));

// Check whether a bit is allocated
if (bitmap[index / 8] & (1 << (index % 8))) {
    /* allocated */
}
```

Recommended helper functions are **set_bit()** and **is_bit_set()**.

12. First-Fit Allocation:

SimpleFS uses first-fit allocation. When an inode or data block is needed, scan from the beginning and allocate the first free one.

```
Inode 1 -> Used
Inode 2 -> Used
Inode 3 -> Free    <-- allocate this one next
Inode 4 -> Free
```

13. Directory Entries and Root Directory:

A directory entry connects a file name to an inode number. Each root-directory entry occupies exactly 64 bytes.

```
typedef struct {
    uint32_t inode_no;
    uint8_t type;
    char name[59];
} dirent_t;
```

Field	Bytes	Purpose
inode_no	4	Inode number; 0 means unused entry
type	1	1 = file, 2 = directory
name	59	File/directory name
Total	64	

A file name may contain at most 58 characters so that the name can be null-terminated inside the 59-byte name field.

```
printf("%zu\n", sizeof(dirent_t)); // must print 64
```

I. Root Directory:

The root directory always uses inode number 1 and data Block 4. Its initial inode values are:

```
type      = 2
links     = 2
size      = 128

direct[0] = 4
direct[1] = 0
direct[2] = 0
```

The initial size is 128 bytes because the root directory initially contains two 64-byte entries: “.” and “..”.

Entry	inode_no	type	name
0	1	2	.
1	1	2	..

Both entries refer to inode 1 because the root directory has no parent.

II. Root Directory Capacity:

```
4096 / 64 = 64 directory entries
64 - 2 special entries = 62 possible user entries
```

However, SimpleFS has only 32 inodes, and inode 1 is reserved for the root. Therefore, the actual maximum is 31 regular files. The root directory will always fit inside one block, so directory expansion is not required.

14. Program 1 - simplefs_builder:

simplefs_builder creates an empty **SimpleFS** image.

```
./simplefs_builder --image disk.img
```

After successful execution, disk.img must exist and contain exactly 262144 bytes.

I. Required Workflow:

- 1) Read the --image command-line argument.
- 2) Create a 262144-byte image and initialize it to zero.
- 3) Initialize the superblock and write it to Block 0.
- 4) Initialize the inode bitmap, set Bit 0, and write it to Block 1.
- 5) Initialize the data bitmap, set Bit 0, and write it to Block 2.
- 6) Create root inode 1 and write it as the first inode in Block 3.
- 7) Create the . and .. directory entries and write them at the beginning of Block 4.
- 8) Close the image safely.

II. TODOs in simplefs_builder.c:

TODO	Required Task
Builder TODO 1	Fill the superblock fields
Builder TODO 2	Allocate the root inode in the inode bitmap
Builder TODO 3	Allocate the root data block in the data bitmap
Builder TODO 4	Initialize the root inode
Builder TODO 5	Initialize the . entry
Builder TODO 6	Initialize the .. entry

15. Program 2 - simplefs_adder:

simplefs_adder adds one regular file from the current working directory into an existing SimpleFS image.

```
./simplefs_adder --input disk.img --file test1.txt
```

The original source file must remain unchanged. Source paths are not required; the source file should be in the current working directory.

I. Required Workflow:

- 1) Open the image in binary read/write mode.
- 2) Read the superblock and verify the magic number 0x53465331.
- 3) Open the source file in binary read mode.
- 4) Determine the source-file size.
- 5) Reject files larger than 12288 bytes.
- 6) Reject file names longer than 58 characters.
- 7) Calculate the number of required data blocks.
- 8) Search the root directory and reject a duplicate file name.
- 9) Read the inode bitmap and find the first free inode.
- 10) Read the data bitmap and allocate the required number of data blocks using first-fit.

- 11) Copy the source-file contents to the allocated blocks.
- 12) Create and write the new file inode.
- 13) Update and write the inode and data bitmaps.
- 14) Create a directory entry for the new file.
- 15) Increase the root inode size by 64 bytes.
- 16) Close the source file and image safely.

II. Calculating Required Blocks:

```
required_blocks = (file_size + BLOCK_SIZE - 1) / BLOCK_SIZE;
```

File Size	Required Blocks
0 bytes	0
100 bytes	1
4096 bytes	1
4097 bytes	2
5000 bytes	2
8192 bytes	2
8193 bytes	3
12288 bytes	3

III. Example: Adding a 5000-Byte File:

If the file is the first regular file added, first-fit allocation should normally allocate inode 2 and Blocks 5 and 6.

```
type      = TYPE_FILE
links     = 1
size      = 5000

direct[0] = 5
direct[1] = 6
direct[2] = 0
```

The first 4096 bytes go to Block 5, and the remaining 904 bytes go to Block 6. Unused bytes in the final block must remain zero.

IV. Updating the Root Directory:

The first two root-directory entries are “.” and “..”. Therefore, the first regular file normally uses Entry 2.

```
inode_no = 2
type     = TYPE_FILE
name     = "test1.txt"
```

Each new directory entry increases the root directory size by 64 bytes. The root size changes from 128 to 192 bytes after the first user file, then to 256 bytes after the second, and so on.

V. TODOs in simplefs_adder.c:

TODO	Required Implementation	Concept
1	Find first free inode	Inode allocation
2	Find first free data block	Data-block allocation
3	Search for duplicate file name	Directory lookup
4	Find free directory entry	Directory organization
5	Calculate required blocks	Block storage
6	Allocate required data blocks	Data bitmap
7	Copy source data into blocks	File data
8	Initialize new file inode	Inode metadata
9	Mark inode allocated	Inode bitmap
10	Create directory entry	Name-to-inode mapping
11	Update root inode size	Directory metadata

16. Important Constraints:

You must carefully distinguish among an inode number, inode-table index, data-bitmap index, and absolute block number.

I. Inode Mapping:

```
Inode 1 -> Table index 0
Inode 2 -> Table index 1
Inode 3 -> Table index 2
```

```
Table index = inode number - 1
```

II. Data-Block Mapping:

```
Data bitmap index 0 -> Absolute Block 4
Data bitmap index 1 -> Absolute Block 5
Data bitmap index 2 -> Absolute Block 6
```

```
Absolute block = DATA_REGION_BLOCK + bitmap index
Bitmap index   = absolute block - DATA_REGION_BLOCK
```

17. Compilation:

Open the terminal in the project directory and compile the two programs as follows:

```
gcc -Wall -Wextra -std=c11 simplefs_builder.c -o simplefs_builder
gcc -Wall -Wextra -std=c11 simplefs_adder.c -o simplefs_adder
```

If compilation succeeds, the executables **simplefs_builder** and **simplefs_adder** should be created. Examine and correct compiler warnings where possible.

18. Executing the Programs:

I. Creating an Image:

```
./simplefs_builder --image disk.img
ls -l disk.img
```

The image must contain exactly 262144 bytes.

II. Adding Files:

```
./simplefs_adder --input disk.img --file test1.txt
./simplefs_adder --input disk.img --file test2.txt
```

19. Examining Correctness:

A program that runs without crashing is not necessarily correct. You must inspect the generated binary image using xxd or hexdump.

```
xxd disk.img

# or

hexdump -C disk.img
```

When starting an independent test, first create a fresh image:

```
xxd disk.img

# or

hexdump -C disk.img
```

I. Test Case 1 - Empty File System:

```
./simplefs_builder --image disk.img
ls -l disk.img
```

Expected: disk.img exists and is exactly 262144 bytes.

II. Test Case 2 - Superblock:

```
xxd -l 128 disk.img
# or
hexdump -C -n 128 disk.img
```

Expected: the beginning of Block 0 contains the binary superblock and is not completely zero.

III. Test Case 3 - Initial Inode Bitmap:

```
xxd -s 4096 -l 16 disk.img
```

Expected: the first byte is 01 because only inode 1 is allocated.

IV. Test Case 4 - Initial Data Bitmap:

```
xxd -s 8192 -l 16 disk.img
```

Expected: the first byte is 01 because only Block 4, the root directory block, is allocated.

V. Test Case 5 - Root Directory:

```
xxd -s 16384 -l 128 disk.img
```

Expected: the ASCII side should show the . and .. directory names.

VI. Test Case 6 - Add a Small File:

```
echo "Operating Systems Lab" > test1.txt
rm -f disk.img
./simplefs_builder --image disk.img
./simplefs_adder --input disk.img --file test1.txt
```

Expected: the file is added successfully.

VII. Test Case 7 - Inode Allocation After One File:

```
xxd -s 4096 -l 8 disk.img
```

Expected: the first byte is 03 (binary 00000011), meaning inode 1 is the root and inode 2 belongs to test1.txt.

VIII. Test Case 8 - Data Allocation After One Small File:

```
xxd -s 8192 -l 8 disk.img
```

Expected: the first byte is 03 because Block 4 is used by the root and Block 5 is used by test1.txt.

IX. Test Case 9 - Verify File Contents:

```
xxd -s 20480 -l 100 disk.img
```

Expected: the ASCII side should contain Operating Systems Lab. This verifies that the source-file data was actually copied to Block 5.

X. Test Case 10 - Two-Block File:

```
dd if=/dev/zero of=test2.txt bs=1 count=5000
rm -f disk.img
./simplefs_builder --image disk.img
./simplefs_adder --input disk.img --file test2.txt
xxd -s 8192 -l 8 disk.img
```

Expected: the data bitmap begins with 07 because Blocks 4, 5, and 6 are allocated. The file inode should use `direct[0] = 5` and `direct[1] = 6`.

XI. Test Case 11 - Maximum File Size:

```
dd if=/dev/zero of=maxfile.dat bs=1 count=12288
dd if=/dev/zero of=too_big.dat bs=1 count=12289
```

The 12288-byte file should be accepted. The 12289-byte file must be rejected with an appropriate error message.

XII. Test Case 12 - Multiple Files:

```
rm -f disk.img
./simplefs_builder --image disk.img
./simplefs_adder --input disk.img --file test1.txt
./simplefs_adder --input disk.img --file test2.txt
./simplefs_adder --input disk.img --file test3.txt
```

Expected under first-fit: inode 1 is the root, and the three user files normally receive inodes 2, 3, and 4.

XIII. Test Case 13 - Duplicate File:

```
./simplefs_adder --input disk.img --file test1.txt
./simplefs_adder --input disk.img --file test1.txt
```

Expected: the second operation is rejected with Error: file already exists in SimpleFS.

XIV. Test Case 14 - Missing Source File:

```
./simplefs_adder --input disk.img --file abc123.txt
```

Expected: Error: source file not found. The program must exit normally.

XV. Test Case 15 - Missing Image:

```
./simplefs_adder --input nothing.img --file test1.txt
```

Expected: Error: file-system image not found. A segmentation fault is not acceptable.

20. Required Error Handling:

- 1) Missing or invalid command-line arguments
- 2) Image cannot be created
- 3) Image does not exist
- 4) Invalid SimpleFS magic number
- 5) Source file does not exist
- 6) Duplicate file name
- 7) File name longer than 58 characters
- 8) File larger than 12288 bytes
- 9) No free inode
- 10) Insufficient free data blocks
- 11) No free directory entry
- 12) File read/write errors where applicable

Programs must display understandable error messages and exit safely. A segmentation fault or abnormal crash is considered incorrect behavior.

21. Common Mistakes:

- 1) **Confusing bitmap index with absolute block number:** Data bitmap index 1 represents absolute Block 5, not Block 1.
- 2) **Confusing inode number with table index:** Inode 2 is stored at table index 1.
- 3) **Forgetting to update a bitmap:** Allocating an inode or data block requires its bitmap bit to be set to 1.
- 4) **Forgetting direct pointers:** If Block 5 is allocated to a file, its inode must contain `direct[0] = 5`.
- 5) **Forgetting the directory entry:** An inode does not store the file name. The root directory must map the file name to its inode number.
- 6) **Storing a bitmap index in `direct[]`:** `direct[]` stores the absolute block number, not the bitmap index.
- 7) **Storing allocated size instead of actual size:** A 5000-byte file has `size = 5000` even though two blocks (8192 bytes) are allocated.
- 8) **Not zero-filling the final block:** If the last block is only partially used, its remaining bytes should be zero.

22. Project Deliverables:

Each group must submit:

```
simplefs.h  
simplefs_builder.c  
simplefs_adder.c  
README.txt
```

README.txt must contain the group number, student IDs, compilation commands, execution examples, a brief description of the implementation, the contribution of each group member, and any known limitations or problems. **Each group must submit the above-mentioned files in a zip file by the deadline. Do not submit executables. Not everyone from a group needs to submit the implementation. A member from a group has to submit the implementation. Further instructions regarding the submission are provided in the submission form.**

23. Compilation Requirement:

```
gcc -Wall -Wextra -std=c11 simplefs_builder.c -o simplefs_builder  
gcc -Wall -Wextra -std=c11 simplefs_adder.c -o simplefs_adder
```

Compilation failure may prevent further testing of the submission.

24. Evaluation Criteria:

The project evaluates both the correctness of the file-system image and the individual understanding of the submitted implementation. The instructor may use input files different from the provided samples.

- 1) Empty image creation and exact image size
- 2) Superblock correctness
- 3) Inode and data bitmap correctness
- 4) Root inode and . / .. entries
- 5) Adding one-, two-, and three-block files
- 6) Adding multiple files
- 7) First-fit inode and data-block allocation
- 8) Directory-entry correctness
- 9) Actual file-data verification
- 10) Duplicate and oversized-file handling
- 11) Missing source/image handling
- 12) Safe error handling

25. Mark Distribution:

Criteria	Marks
Compilation and Command-Line Handling	5
Correct File-System Image Creation	5
Superblock Implementation	5
Inode and Data Bitmaps	5
Root Inode and Root Directory	5
Adding a One-Block File	5
Adding a Two/Three-Block File	5
Error Handling and Overall Correctness	5
Individual Viva	60
Total	100

26. Academic Integrity Requirement:

The objective is to understand file system implementation, not merely to produce a working executable. Every group member must understand the superblock, inode, inode bitmap, data bitmap, directory entry, direct block pointers, first-fit allocation, block-address calculations, and file-creation process. A working program without sufficient individual understanding will not receive full credit.

27. Source Files:

The necessary source files for the project can be found in [this folder](#).

28. Submission Guidelines:

Can be found in the submission form given below.

29. Lab Term Project Walkthrough File:

An interactive [HTML file](#) titled “**Summer 2026: CSE321 Lab Term Project Walkthrough**” is provided with the project materials. Please download and go through this walkthrough before starting the project. It explains the SimpleFS project step by step, including the file system structure, TODO tasks, implementation process, compilation, execution, and testing.

The walkthrough is provided to help you understand and implement the project more easily. However, always follow the project specification and provided skeleton files for the final implementation and submission.

Deadline: August 28, 2026, 11:50 PM

Submission Form Link:

<https://forms.gle/5wUx2kQ4EMDpyntE9>

(Submissions outside the Google Form will be automatically rejected.)